

Project Development Overview

The Personalized Networking Assistant was developed using a modular architecture that separates the frontend, backend, AI services, and data storage. The frontend was implemented using Streamlit to provide an interactive user interface, while FastAPI was used to develop RESTful APIs for processing user requests. The application integrates DistilBERT for event theme extraction, Gemini for AI-powered conversation generation, and the Wikipedia API for fact checking. Conversation history and user feedback are stored in JSON files, ensuring persistent storage throughout the application lifecycle.

Location of Source Code

The complete source code is maintained in a GitHub repository and is organized into separate frontend and backend modules for better maintainability.

Repository Name:
personalisedAIAssistant

GitHub Repository:
https://github.com/Yoshita-22/Personalised_ai_Assistant

Source Code Overview

The project follows a modular directory structure where each component is responsible for a specific functionality.

Module	Description
frontend/app.py	Implements the Streamlit user interface for conversation generation, fact checking, history, and feedback.
backend/app/main.py	Entry point of the FastAPI application and registers API routes.
backend/app/routers/conversation.py	Defines API endpoints and coordinates communication between frontend and service modules.

backend/app/services/event_analyser.py	Uses DistilBERT Zero-Shot Classification to extract event themes from user input.
backend/app/services/topic_generator.py	Uses Gemini to generate personalized networking conversation starters.
backend/app/services/fact_checker.py	Retrieves relevant information from Wikipedia for fact verification.
backend/app/services/history_logger.py	Stores and retrieves conversation history using history.json .
backend/app/services/feedback_logger.py	Records user feedback (Like/Dislike) in feedback.json .
backend/app/schemas/request_response.py	Defines Pydantic request and response models for API validation.
backend/app/data/history.json	Stores generated conversation sessions.
backend/app/data/feedback.json	Stores user feedback history.
backend/tests/	Contains unit tests for API routes and service modules using Pytest